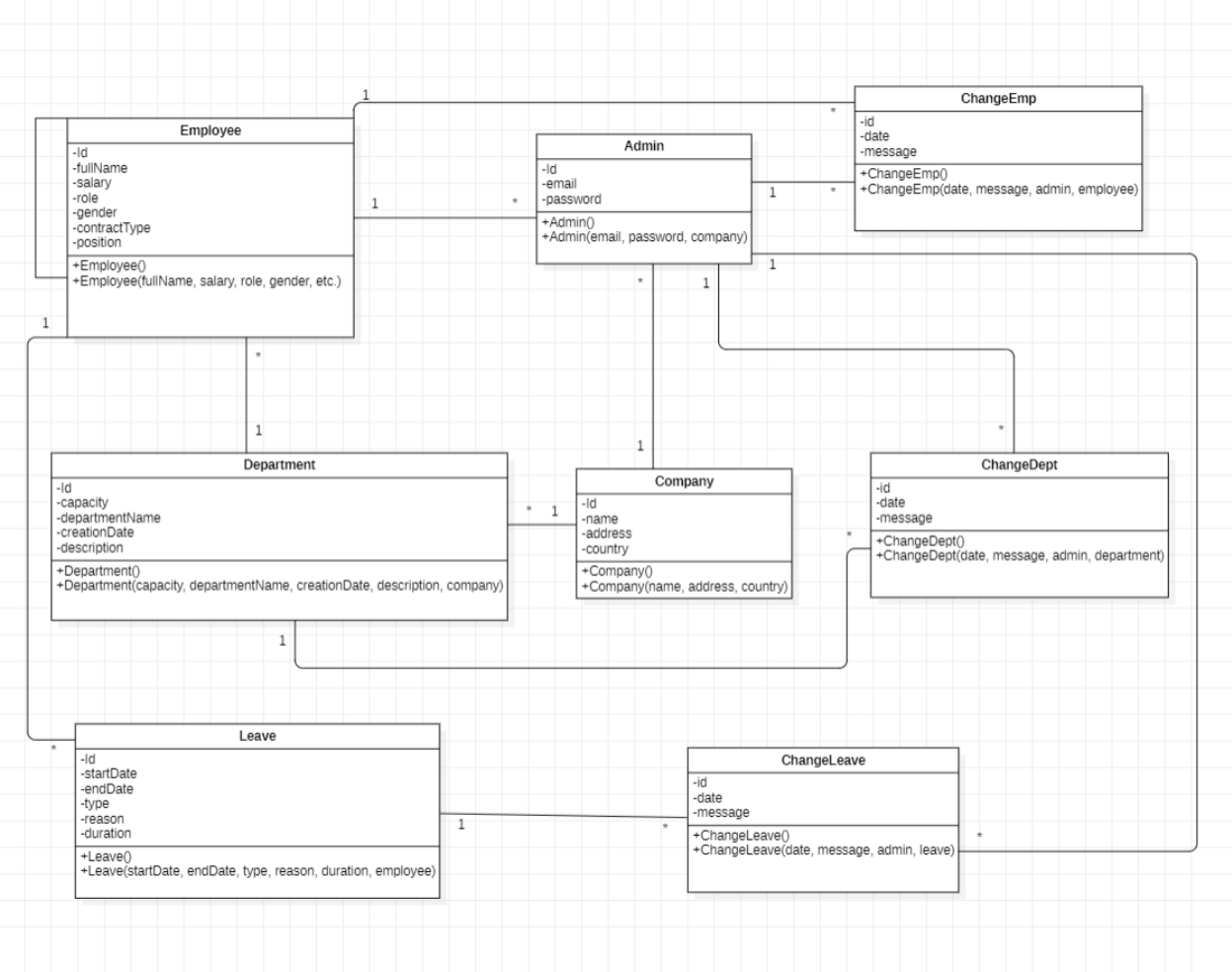
Class Diagram Assignment:



Class Diagram Description: Employee Management System

This class diagram models the core domain classes required to implement the Employee Management System, focusing on the static structure and the relationships between different objects within the application.

Core Domain Classes:

- **Company:** Acts as the central structural class. It holds basic company details (name, address, country) and maintains one-to-many associations with the Department, Employee, and Admin classes.

- **Admin:** Represents the system administrators who manage the application. The Admin class is responsible for executing modifications to the system, which generates corresponding change log objects.
- **Department:** Represents the organizational teams within the company. Each Department object is linked to the Company and maintains a collection of Employee objects (a 1-to-many relationship).
- **Employee:** Stores individual staff details (salary, role, contract type, etc.). Each Employee instance belongs to exactly one Department and one Company. The Employee class also contains a recursive relationship to represent the supervisory hierarchy, where one Employee (the supervisor) can manage multiple other Employees.
- **Leave:** Represents time-off records (tracking start/end dates, type, and reason). Each Leave object is tied to exactly one Employee, while an Employee can have multiple Leave records over time.

Change Log (Associative) Classes: To track system modifications and maintain an audit trail, the model includes three specific change-tracking classes: **ChangeEmp**, **ChangeDept**, and **ChangeLeave**.

- Whenever an Admin modifies an entity, an instance of the respective change class is created.
- These classes store a Date and a descriptive Message regarding the modification.
- Each change object acts as a link between the Admin who made the change and the specific Employee, Department, or Leave object that was altered.

Navigation and Cardinality: The diagram primarily utilizes standard one-to-many (1 to *) associations. For example, one Company instance connects to many Department instances, and one Admin can generate many ChangeEmp log instances. These associations guide how the application's code will reference objects, such as utilizing Java List collections to handle the "many" sides of the relationships.